

E4-RESSOURCE1 : PSEUDOCODE – PYTHON – C/C++

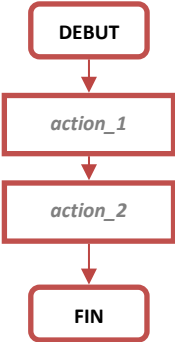
OPÉRATEURS

Nature	Type	Notation				Signification	
		Pseudocode	Python		C/C++		
Opérateur d'affectation	Tous types	←	=		=	Affectation (Prend la valeur de...)	
Opérateurs arithmétiques	Entier Réal	+	+		+	Addition	
		-	-		-	Soustraction	
		*	*		*	Multiplication	
		/	/		/	Division réelle	
		DIV	//		/	Division entière	
		MOD (Modulo)	%		%	Reste d'une division entière	
Opérateurs logiques	Booléen	ET	and		&& and	ET logique	
		OU	or		or	OU logique	
		NON	not		! not	NON logique	
Opérateur bit à bit	Entier	ET bit à bit	&		& bitand	ET bit à bit	
		OU bit à bit			bitor	OU bit à bit	
		⊕	^		^ xor	OU EXCLUSIF bit à bit	
		Décalage à gauche de <i>n</i>	<< <i>n</i>		<< <i>n</i>		Décalage à gauche de <i>n</i> bits
		Décalage à droite de <i>n</i>	>> <i>n</i>		>> <i>n</i>		Décalage à droite de <i>n</i> bits
Opérateur de concaténation	Chaîne de caractères	+	+		+	Concaténation	
Opérateurs de comparaison	Tous types	=	==	is	==		Égal
		≠	!=	is not	!=	not_eq	Différent
		<	<		<		Inférieur
		>	>		>		Supérieur
		≤	<=		<=		Inférieur ou égal
		≥	>=		>=		Supérieur ou égal

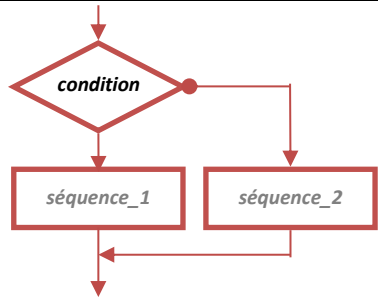
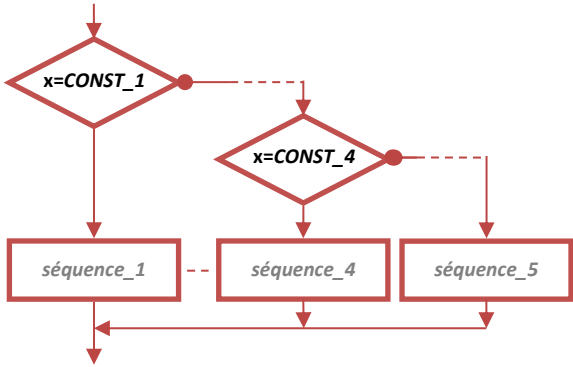
VARIABLES ET CONDITIONS

		Pseudocode	Python	C/C++
Variable	Déclaration	variable : <i>type</i>	Le typage est dynamique, il s'effectue lors de l'affectation de la variable. On peut représenter le typage sous la forme d'un commentaire : #Typage dynamique # <i>variable</i>: <i>type</i> Il est possible d'annoter le typage avec la syntaxe suivante, on utilisera cette solution : #Annotation de type variable: <i>type</i>	<i>type</i> variable;
		i: entier f: réel	i: int f: float	int i; float f;
		Affectation	variable ← <i>valeur</i> variable ← <i>calcul</i>	variable = <i>valeur</i> variable = <i>calcul</i>
	i1 ← 10 i2 ← 4*i1/2		i1 = 10 i2 = 4*i1/2	i1 = 10; i2 = 4*i1/2;
	Déclaration avec affectation	variable: <i>type</i> ← <i>valeur</i>	i: <i>type</i> = <i>valeur</i>	// syntaxe 1 <i>type</i> variable = <i>valeur</i> ; // syntaxe 2 <i>type</i> i (<i>valeur</i>);
		i: entier ← 10	i: int = 10	// syntaxe 1 int i = 10; // syntaxe 2 int i (10);
	Constante	Déclaration	VAR: <i>type</i> ← <i>valeur</i>	VAR: <i>type</i> = <i>valeur</i>
PI: réel ← 3.14			PI: float = 3.14	// syntaxe 1 const double PI = 3.14; // syntaxe 2 const double PI (3.14);
Condition		i = 10 ET j ≥ 30	(i == 10) and (j >= 30)	(i == 10) && (j >= 30)

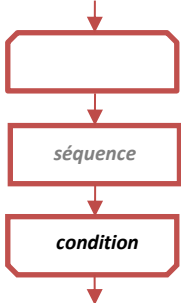
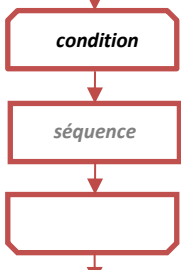
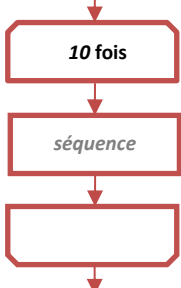
STRUCTURES DE BASE, LECTURE, ÉCRITURE

Algorithme	Pseudocode	Python	C/C++
 <pre> graph TD DEBUT --> action_1 action_1 --> action_2 action_2 --> FIN </pre>	DÉBUT <i>action_1</i> <i>action_2</i> FIN	# DÉBUT <i>action_1</i> <i>action_2</i> # FIN	<pre> int main() { action_1; action_2; return 0; } </pre>
	afficher ("Hello world!")	<pre>print("Hello world!")</pre>	<pre> #include <iostream> cout << "Hello world!" << endl; </pre>
	$x \leftarrow \text{saisir}()$	<pre> x = input() x = int(input()) x = input("texte avant saisie") </pre>	<pre> #include <iostream> int x; cin >> x; </pre>
	PROCEDURE (arg1: entier, arg2: réel) DÉBUT <i>séquence</i> FIN	<pre>def procedure(arg1, arg2): séquence</pre>	<pre>void procedure(int arg1, float arg2) { séquence; }</pre>
	FONCTION (arg1: entier, arg2: réel): entier DÉBUT <i>séquence</i> ..retourner <i>valeur</i> FIN	<pre>def fonction(arg1, arg2): séquence ..return valeur</pre>	<pre>int fonction(int arg1, float arg2) { séquence; ..return valeur; }</pre>

STRUCTURES CONDITIONNELLES

Algorithme	Pseudocode	Python	C/C++
	SI <i>condition</i> ALORS <i>séquence_1</i> SINON <i>séquence_2</i> FIN_SI	if <i>condition</i> : <i>séquence_1</i> else : <i>séquence_2</i>	if (<i>condition</i>) { <i>séquence_1</i> ; } else { <i>séquence_2</i> ; }
	SELON <i>x</i> CAS <i>CONST_1</i> <i>séquence_1</i> ... CAS <i>CONST_4</i> <i>séquence_4</i> AUTRE_CAS <i>séquence_5</i> FIN_SELON	if <i>x</i> == <i>CONST_1</i> : <i>séquence_1</i> ... elif <i>x</i> == <i>CONST_4</i> : <i>séquence_4</i> else : <i>séquence_5</i>	switch (<i>x</i>) { case <i>CONST_1</i> : <i>séquence_1</i> ; break ; ... case <i>CONST_4</i> : <i>séquence_4</i> ; break ; default : <i>séquence_5</i> ; }

STRUCTURES ITÉRATIVES

Algorithme	Pseudocode	Python	C/C++
 <pre> graph TD Start(()) --> Seq[séquence] Seq --> Cond[condition] Cond --> Seq </pre>	FAIRE <i>séquence</i> TANT_QUE <i>condition</i>	Cette boucle n'existe pas en Python, vous pouvez la simuler avec la structure suivante : <pre> while True: séquence if not condition: break </pre>	<pre> do { séquence; } while(condition); </pre>
 <pre> graph TD Start(()) --> Cond[condition] Cond --> Seq[séquence] Seq --> Empty[] Empty --> Cond </pre>	TANT_QUE <i>condition</i> [FAIRE] <i>séquence</i> FIN_TANT_QUE	<pre> while condition : séquence </pre>	<pre> while(condition) { séquence; } </pre>
 <pre> graph TD Start(()) --> Rep[10 fois] Rep --> Seq[séquence] Seq --> Empty[] Empty --> Rep </pre>	POUR <i>i</i> DE 0 À 9 [RÉPÉTER] <i>séquence</i> FIN_POUR	<pre> for i in range(0, 10) : séquence </pre>	<pre> for(int i=0 ; i<10 ; i++) { séquence; } </pre>

PROCÉDURES, FONCTIONS

Pseudocode	Python	C/C++
PROCÉDURE nom_procédure(arg1: type, arg2: type, ...) DÉBUT /* variable */ FIN	<pre>def nom_procédure(arg1: type, arg2: type, ...) -> None : # DÉBUT # FIN</pre>	<pre>void nom_procédure(type arg1, type arg2, ...) { ...; ...; }</pre>
FUNCTION nom_fonction(arg1 : type, arg2: type, ...): type_résultat DÉBUT /* variables */ renvoyer nom_variable (ou calcul) FIN	<pre>def nom_fonction(arg1: type, arg2: type, ...) -> type_résultat : # DÉBUT return nom_variable # FIN</pre>	<pre>type_résultat nom_fonction(type arg1, type arg2, ...) { ...; ...; return nom_variable; }</pre>

TABLEAUX

Tableau	Pseudocode	Python	C/C++
Déclaration	nom_tableau: tableau [taille] de types	nom_tableau: type = [0]*taille	type nom_tableau [taille];
	tab: tableau [4] d'entiers	tab: int = [0]*4 #tab = [0, 0, 0, 0] tab: int = range(0, 5) #tab = [0, 1, 2, 3, 4]	int tab [4];
Déclaration et affectation	nom_tableau: tableau [taille] de types ← [valeur_1, valeur_2, ...]	nom_tableau: type = [valeur_1, valeur_2, ...]	types nom_tableau [taille] = {valeur_1, valeur_2, ...};
	tab: tableau [4] d'entiers ← [3, 0, 8, 2]	tab: int = [3, 0, 8, 2]	int tab [4] = {3, 0, 8, 2};
Affectation d'une case	nom_tableau[indice] ← valeur nom_tableau[indice] ← calcul	nom_tableau[indice] = valeur nom_tableau[indice] = calcul	nom_tableau[indice] = valeur; nom_tableau[indice] = calcul;
	tab[4] ← 10 tab[2] ← 4*x/2	tab[4] = 10 tab[2] = 4*x/2	tab[4] = 10; tab[2] = 4*x/2;

CHAÎNES DE CARACTÈRES

Chaîne	Pseudocode	Python	C/C++
Déclaration	nom_chaine: chaîne	nom_chaine: str	string nom_chaine; // C++ char* nom_chaine; // C
	txt: chaîne	txt: str	string txt; #include <string>
Déclaration et affectation	nom_chaine: chaîne ← "texte"	txt: str = "texte"	string nom_chaine = "texte";
	txt : chaîne ← "Hello world!"	txt: str = "Hello world!"	string txt = "Hello world!"; string txt ("Hello world!");
Accès à un caractère	nom_chaine[<i>indice</i>] ← valeur	Une chaîne est <u>immuable</u> en Python.	nom_chaine[<i>indice</i>] = valeur;
	txt[0] ← 'h'	Il faut utiliser la méthode : nom_chaine. replace()	txt[0] = 'h';